NRLC-06: SLO & Workload Migration

> **Series:** NRLC — New Relic to Dynatrace Migration Deep Dives | **Notebook:** 6 of 9 | **Created:** April 2026 | **Last Updated:** 10/14/2026

Overview

SLOs are the service contracts that survive migration only if their math survives. Workloads are the groupings that determine what an SLO targets. This deep dive covers SLO metric expression migration, what the `audit-slos` command does and does not check (it validates the Dynatrace side only; the NR-vs-DT math comparison is manual), and the conversion of NR workloads to Gen3 OpenPipeline enrichments + IAM-scoped buckets.

Phase 23 key transactions: `key_transaction_transformer` auto-emits a bundle for each NR Key Transaction — Platform SLO (SLO Service Public API, `/platform/slo/v1/slos`) + OpenPipeline enrichment attribute (`keyTransaction.name`) + Workflow with `migratedFrom` tag. Phase 11 `slo_transformer` handles SLO v1 / v2 and emits Platform SLO API bodies — `dynatrace_platform_slo` in the Terraform export, Monaco `slo-v2` in the Monaco export; the classic `builtin:monitoring.slo` schema is no longer emitted (`main` branch, read 10/14/2026); Phase 11 `workload_transformer` emits an OpenPipeline enrichment rule + Grail Filter Segment (`dynatrace_segment`; not a `builtin:*` Settings schema) + bucket-scoped IAM policy (Gen3 default) with `LegacyWorkloadTransformer` preserving Management Zone under `--legacy`.

Table of Contents

1. SLO Models Compared
2. Metric Expression Migration
3. SLO Auditor
4. Workload → OpenPipeline Enrichment + IAM Policy
5. Entity Selector Translation
6. Validation — 7-Day SLI Delta

Prerequisites

Requirement	Details
Audience	SRE leads, service owners maintaining SLOs
Standalone	This notebook is self-contained for SLO + workload migration. No required prerequisite reading.
Optional depth	NRLC-02 (full NRQL→DQL compiler), NRLC-08 (validation), NRLC-09 (toolchain)

Requirement	Details
Stakes	SLOs drive SLA reporting; arithmetic mismatch causes compliance issues

Embedded Translation Context — SLO Indicator Patterns

Every SLO contains a NRQL indicator query that must produce the equivalent DQL. The patterns below cover the vast majority of SLO indicators.

Availability SLO

```
-- NRQL: "good" events / total
SELECT percentage(count(*), WHERE error IS FALSE) FROM Transaction
-- target: &gt; 99.9
```

```
// DQL (SLI): share of requests that did not fail, per service
timeseries {total = sum(dt.service.request.count), failed =
sum(dt.service.request.failure_count)}, from:-1h, by:{dt.smartscape.service}
| fieldsAdd sli = 100 * (1 - arraySum(failed) / arraySum(total))
| fieldsAdd service = getNodeName(dt.smartscape.service)
| fields service, sli
// target: 99.9
```

Base availability on the service request metrics (or on `span.status_code == "error"` for span-level SLIs), not on `error`. `error` is not a Dynatrace field: `isNull(error)` treats every request without a custom `error` attribute as good, and on the validation tenant it read 97.8% where `span.status_code` gave 96.2% over the same request spans (10/05/2026) — an SLO that reads healthier than reality.

Latency SLO

```
-- NRQL
SELECT percentage(count(*), WHERE duration &lt; 0.5) FROM Transaction
-- target: &gt; 99
```

```
fetch spans, from:-1h
| filter isNotNull(endpoint.name)
| summarize sli = 100.0 * countIf(duration &lt; 500ms) / count()
// target: 99
```

Error budget burn rate

```
-- NRQL
SELECT count(*) FROM TransactionError SINCE 1 hour ago
-- threshold: above (1 - SLO_target) * total
```

```
fetch spans, from:-1h
| filter isNotNull(endpoint.name)
| summarize error_rate = 100.0 * countIf(span.status_code == "error") / count()
// target inverted: error budget = 100 - SLO target
```

Translation Confidence for SLOs

Indicator pattern	Confidence	Notes
Percentage-based SLI	HIGH	Direct
Threshold-based with `duration`	HIGH	Use a duration literal (`duration < 500ms`); `duration("500ms")` does not parse
Error budget burn	MEDIUM	Verify alignment of denominator
Multi-source SLI (joined data)	LOW	Manual review

1. SLO Models Compared

NR SLO Model

An NR SLO has:
- `target` (e.g., 99.9%)
- `timeWindow` (rolling 7d, 28d, etc.)
- `indicator` (a NRQL query computing the SLI value)
- `threshold` (the value the SLI must satisfy)
- entities the SLO targets (via workload or entity selector)

DT SLO Model (Service-Level Objectives app)

The migration target is the Grail-based **Service-Level Objectives** app, reached through the **SLO Service Public API** (Terraform: `dynatrace_platform_slo`). A DT SLO has:
- `name` , `description` , `tags`
- `criteria` — `target` (numeric percentage), optional `warning` (early-warning percentage), and the evaluation timeframe as `timeframeFrom` / `timeframeTo` (e.g. `now-7d` / `now`)
- `customSli.indicator` — a DQL query that returns the SLI as a percentage
- optional filter segments that scope the SLI (bucket + enriched-attribute scope in Gen3)

Structurally similar; the math layer is what migrates.

> **Not the classic SLO.** `builtin:monitoring.slo` (Terraform `dynatrace_slo_v2` , API token scopes `slo.read` / `slo.write`) is **SLO Classic**. Use it only to manage classic SLOs that already exist; author migrated SLOs in the modern app, which authenticates with platform permissions (`slo:slos:read` / `slo:slos:write`). SLO-05 covers both paths and the classic surface's upgrade status. Dynatrace's upgrade guidance for existing classic SLOs is to *"keep both SLOs running for one evaluation period and compare the values before you remove the Classic SLO."*

> **Sources:** Upgrade from Service-Level Objectives Classic (DT docs), dynatrace_platform_slo resource (Dynatrace GitHub) — *"an OAuth client including the following permissions: View SLOs (`slo:slos:read`) and Create and edit SLOs (`slo:slos:write`)"*, dynatrace_slo_v2 resource (Dynatrace GitHub) — "schemaId: `builtin:monitoring.slo` ".

2. Metric Expression Migration

The SLI is a fraction:

```
SLI = good_events / total_events
```

Both NR and DT model this; the difference is how each side counts.

Example — Availability SLO

NR NRQL indicator:

```
SELECT count(*) FROM Transaction WHERE error IS FALSE
```

NR threshold: result / total > 0.999

DT SLI (DQL):

```
// DQL (SLI): share of requests that did not fail, per service
timeseries {total = sum(dt.service.request.count), failed =
sum(dt.service.request.failure_count)}, from:-1h, by:{dt.smartscape.service}
| fieldsAdd sli = 100 * (1 - arraySum(failed) / arraySum(total))
| fieldsAdd service = getNodeName(dt.smartscape.service)
| fields service, sli
// target: 99.9
```

DT target: 99.9

Example — Latency SLO

NR NRQL indicator:

```
SELECT percentage(count(*), WHERE duration &lt; 0.5) FROM Transaction
```

DT SLI (DQL):

```
fetch spans, from:-1h
| filter isNotNull(endpoint.name)
| summarize sli = 100.0 * countIf(duration &lt; 500ms) / count()
// target: 99
```

Conversion considerations:

- NR units in NRQL are often seconds; DT durations take a duration literal (`500ms`). To turn a duration into a number, divide by a unit (`duration / 1ms`)

- NR's `error IS FALSE` maps to "request not failed": the `dt.service.request.failure_count` metric, or `span.status_code != "error"` on request spans. `isNull(error)` is not a translation — `error` is not a Dynatrace field
- Time windows: NR `SINCE 7 days ago` becomes the SLO criteria timeframe `timeframeFrom: now-7d`, `timeframeTo: now`
- Entity scope: NR's workload becomes a DT bucket + enriched-attribute filter (Gen3 pattern)

3. SLO Auditor

The `Dynatrace-NewRelic` project includes an **SLO auditor** (`registry/slo_auditor.py`, run as `migrate.py audit-slos`). It checks the **Dynatrace side only**. The command's docstring describes it as an audit of the SLOs in the Dynatrace environment for metric validity: it reads the tenant's SLOs, checks whether each is evaluating, and validates every metric its DQL references against the tenant's metrics — missing metrics, invalid aggregations, and NRQL syntax that was not converted. It does not query New Relic, so it cannot tell you whether a DT SLI matches the NR one.

```
# Requires DYNATRACE_ENVIRONMENT_URL and DYNATRACE_OAUTH_TOKEN; exits if either is missing
python3 migrate.py audit-slos
```

It prints an *SLO Audit Results* table — one row per SLO with an OK / FAIL status and the issues found — followed by the number of valid SLOs.

Math equivalence is a manual step. For each migrated SLO, run the original NRQL indicator in New Relic and the DQL SLI in Dynatrace over the same window, and compare the two values (§6).

4. Workload → OpenPipeline Enrichment + IAM Policy

NR workloads are user-defined entity collections, scoped by entity tags or explicit GUID lists.

In **Dynatrace Gen3**, the equivalent is built on **OpenPipeline enrichment** — ingested data is enriched with workload-identifying attributes at ingest (e.g., `workload.name`, `team.owner`, `environment`), and **IAM policies** + **bucket routing** scope access by those enriched attributes. This is fundamentally different from the Gen2 management-zone or auto-tagging approach (both of which scope entity *visibility*); the Gen3 model scopes the *data itself* via attributes that travel with every record.

NR Workload Field	Gen3 Equivalent
`name`	OpenPipeline enrichment value (e.g., `workload.name = "checkout"`)
`entityGuids[]`	OpenPipeline matcher resolves the entity by name/host group and emits the enriched attribute
`entitySearchQueries[]` (NRQL filter)	OpenPipeline matcher conditions (matchesValue on namespace, host group, etc.)
Tags-based selection	OpenPipeline matcher reads existing tags and emits a normalized workload attribute

The `WorkloadTransformer` emits one OpenPipeline enrichment rule per workload (plus an optional bucket-routing rule if the workload should land in its own bucket) and a recommended IAM policy condition (e.g., `storage:bucket-name == "workload_checkout_logs"`). Once enriched, the workload identifier is queryable in DQL (`filter workload.name == "checkout"`) and durable across dashboards, alerts, and SLOs without depending on Gen2 entity grouping.

5. Entity Selector Translation

DT entity selectors are the language for entity rules. Common patterns:

NR Concept	DT Entity Selector
All hosts in account	`type(HOST)`
Hosts tagged `env:prod`	`type(HOST),tag("env:prod")`
Specific service by name	`type(SERVICE),entityName("checkout-service")`
Services in a namespace	`type(SERVICE),fromRelationships.runsOn(type(KUBERNETES_NAMESPACE),entityName("prod"))`
All entities in a workload	`tag("workload:<name>")` (Gen2 entity tag) **or** Gen3 DQL filter `filter workload.name == "<name>"` (attribute set by OpenPipeline enrichment)

Recommendation: prefer **entity selectors over GUIDs** in any migrated artifact (dashboards, alerts, SLOs). Selectors are portable across tenants; GUIDs are not.

6. Validation — 7-Day SLI Delta

Over a 7-day window, run each SLO's NRQL indicator in New Relic and its DQL SLI in Dynatrace, and compare. This is manual — `audit-slos` checks only that the DT SLOs evaluate and reference real metrics. Acceptance criteria:

- Per-SLO delta ≤ 0.5% (tunable per service-criticality)
- All SLOs report a value (no MISSING_DATA results)
- Bucket + enriched-attribute scope returns the same entity count in both platforms (±5%)

When delta exceeds tolerance:

1. Compare the values day by day — is the delta consistent (suggests a math bug) or sporadic (suggests data-availability issues)?
2. Check unit conversion (ms vs. s vs. ns)

3. Check entity scope — is the bucket / OpenPipeline enrichment filter capturing the same set as NR's workload?
4. Check time alignment — NR's `SINCE 7 days ago` may bin differently than DT's `from:-7d`

Summary

SLO migration succeeds when the math is equivalent within tolerance. `audit-slos` confirms the Dynatrace SLOs evaluate and reference real metrics; the NR-vs-DT comparison itself is a manual check. workload → OpenPipeline enrichment conversion is mechanical for tag-based workloads and manual for GUID-based ones. Always run the 7-day delta check before declaring SLO migration complete.

Continue to **NRLC-07 Logs, Tags & Drop Rules**.

Tooling for SLO-Only Migration

```
# 1. Inventory NR SLOs and Workloads
python3 migrate.py migrate --export-only --components slos,workloads --output ./slo-run

# 2. Translate metric expressions; emit SLO + OpenPipeline enrichment definitions and a report — nothing is imported
python3 migrate.py migrate --dry-run --report --components slos,workloads --output ./slo-run

# 3. Import (export → transform → import); writes ./slo-run/rollback-manifest.json
python3 migrate.py migrate --components slos,workloads --output ./slo-run

# 4. Validate the Dynatrace SLOs: evaluating, metrics exist (DT side only — needs DYNATRACE_OAUTH_TOKEN)
python3 migrate.py audit-slos

# 5. Compare NR and DT SLI values over the same 7-day window by hand (§6)
```

> **Flags checked against `migrate.py` on the tool's `main` branch (10/05/2026).** There is no `--transform-only`: `--dry-run` runs export and transform, writes `<output>/transformed/dynatrace_config.json`, and skips the import. `--diff` takes effect only on a full or dry run — with `--import-only` it is ignored, and without `--dry-run` it is computed *after* the import. `--import-only` needs `--input <dir>` and does not write a rollback manifest; a full run writes `<output>/rollback-manifest.json`. Run `python3 migrate.py migrate --help` against your checkout before relying on a flag.

Note that `slos` is declared as depending on `alerts` (which depends on `notification_channels`), and the tool adds dependencies automatically — so `--components slos,...` also exports, transforms and imports alerts and notification channels. Check the step 2 report before step 3.

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source NewRelic-to-Dynatrace-Migration-Utilities, nrql-engine (planned future home: the `dynatrace-dma` Dynatrace Migration Assistant organization),

and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.